

Future-Regime Supervised Dataset Generator

Complete Installation, Configuration, Generation, Validation, and Troubleshooting Manual

Forecasting task: use the image of candles t to $t+29$ to predict the cluster assigned to candles $t+30$ to $t+59$.

1. What the package does

The package builds a supervised learning dataset from a chronological OHLC price file. For every valid starting position t , it creates a 30-candle input image from candles t through $t+29$. It then creates the following 30-candle target image from candles $t+30$ through $t+59$. The target image is passed through the already trained unsupervised pipeline: frozen pretrained ResNet-18, saved PCA, and saved four-cluster K-Means. The resulting future cluster ID becomes the supervised target for the current image.

`input_image(t..t+29) -> target_cluster_id of target_image(t+30..t+59)`

The future target image is used only during dataset construction. When the supervised classifier is trained later, it should receive only `input_image` and learn to predict `target_cluster_id`.

2. Package contents

File	Purpose
<code>generate_future_regime_dataset.py</code>	Main generator: loads OHLC, creates images, labels future windows, splits data, and writes reports.
<code>validate_generated_dataset.py</code>	Checks manifests, missing files, cluster counts, and chronological leakage conditions.
<code>config.json</code>	All user-editable paths, column names, chart settings, split fractions, and runtime settings.
<code>requirements.txt</code>	Python dependencies.
<code>install_requirements.bat</code>	Windows helper that installs dependencies.
<code>run_generator.bat</code>	Windows helper that runs the main generator.
<code>validate_dataset.bat</code>	Windows helper that runs validation.
<code>README.md</code>	Short project overview.
<code>QUICK_START.txt</code>	Brief setup checklist.

3. Required inputs

- The original chronological Bitcoin OHLC CSV, preferably hourly and ordered by timestamp.
- The saved `pca.joblib` from the completed autoscaled unsupervised experiment.
- The saved `kmeans.joblib` from the same experiment.
- Python 3.10 or newer is recommended.
- Internet access for the first run if the pretrained ResNet-18 weights are not already cached.
- Enough free disk space for thousands of 360 x 360 JPEG images and CSV reports.

4. Recommended folder arrangement

```
C:\Users\m0zim\Desktop\future_regime_dataset_package\
├── generate_future_regime_dataset.py
├── validate_generated_dataset.py
├── config.json
├── requirements.txt
├── install_requirements.bat
├── run_generator.bat
├── validate_dataset.bat
├── models\
│   ├── pca.joblib
│   └── kmeans.joblib
```

The PCA and K-Means files may be stored anywhere, but placing them in a models folder inside the package makes the configuration easier. For faster image generation, a normal local folder is preferable to a heavily synchronized OneDrive folder.

5. Step 1 - Extract the package

1. Download future_regime_dataset_package.zip.
2. Right-click the ZIP and choose Extract All.
3. Open the extracted future_regime_dataset_package folder.
4. Confirm that generate_future_regime_dataset.py and config.json are present.

6. Step 2 - Install Python dependencies

Option A: double-click installation

Double-click install_requirements.bat. A Command Prompt window will install the packages and remain open when finished.

Option B: install from Command Prompt or PowerShell

```
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

The required packages are NumPy, pandas, Matplotlib, Pillow, scikit-learn, Joblib, PyTorch, and Torchvision.

Optional virtual environment

```
python -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt
```

Activation is not required. The virtual environment Python can be called directly, which avoids PowerShell execution-policy problems.

7. Step 3 - Edit config.json

Open config.json in Notepad, IDLE, VS Code, or another text editor. The most important settings are the four absolute paths.

```
"ohlcv_csv": "C:\\Users\\m0zim\\Desktop\\data\\btc_1h.csv",
"pca_model": "C:\\Users\\m0zim\\Desktop\\future_regime_dataset_package\\models\\pca.joblib",
"kmeans_model": "C:\\Users\\m0zim\\Desktop\\future_regime_dataset_package\\models\\kmeans.joblib",
"output_dir": "C:\\Users\\m0zim\\Desktop\\supervised_future_regime_dataset"
```

JSON uses backslashes as escape characters, so Windows paths must contain double backslashes, as shown above.

7.1 Confirm the OHLC column names

```
"columns": {
  "timestamp": "timestamp",
  "open": "open",
  "high": "high",
  "low": "low",
  "close": "close"
}
```

Change these values if the CSV uses names such as Date, datetime, Open, High, Low, Close, or open_time. Column matching is case-insensitive for common names, but explicit correct names are safer.

7.2 Frequency and sample definition

```
"expected_frequency": "1h",
"window_size": 30,
"forecast_horizon": 30,
"stride": 1
```

- expected_frequency="1h" means every adjacent timestamp must be exactly one hour apart.
- window_size=30 creates the current 30-candle input chart.
- forecast_horizon=30 creates the next 30-candle target chart.
- stride=1 creates a new sample at every candle, so adjacent samples overlap heavily.

7.3 Chart style

```
"chart_style": {
  "width_px": 360,
  "height_px": 360,
  "dpi": 100,
  "candle_width": 0.62,
  "wick_width": 1.0,
  "bullish_color": "#00A000",
  "bearish_color": "#D00000",
  "background_color": "white",
  "y_margin_fraction": 0.05,
  "x_margin": 0.7
}
```

These defaults generate 360 x 360, per-window autoscaled candlestick images with hidden axes. Because the original autoscaled generator source was not fully recovered, verify that the produced

charts visually match the earlier unsupervised dataset before launching the full run. A style mismatch can change ResNet embeddings and therefore produce unreliable pseudo-labels.

7.4 Chronological split

```
"split": {
  "train_fraction": 0.70,
  "validation_fraction": 0.15,
  "test_fraction": 0.15,
  "extra_embargo_candles": 0
}
```

The generator does not randomly split samples. It creates chronological partitions and purges rows near the boundaries until the final training target ends before the first validation input begins, and the final validation target ends before the first test input begins.

`extra_embargo_candles` adds an additional gap beyond the required non-overlap. A value such as 30 can be used for a more conservative separation, but it reduces the number of usable samples.

7.5 Runtime settings

```
"device": "auto",
"batch_size": 32,
"num_workers": 0,
"cpu_threads": 4,
"random_seed": 42,
"overwrite_output": false
```

- `device="auto"` uses CUDA if available; otherwise it uses the CPU.
- `batch_size=32` is a safe starting value. Reduce it if GPU memory is insufficient.
- `num_workers=0` is the safest Windows setting.
- `overwrite_output=false` prevents accidental deletion of an existing dataset.

8. Step 4 - Perform a small test run first

A full run can create many thousands of images. Before committing to it, test the package with a smaller chronological OHLC file, for example the first 500 to 2,000 rows. This verifies paths, columns, timestamp continuity, image style, model loading, and output structure.

1. Create a temporary small CSV containing the same columns as the full OHLC file.
2. Change `ohlc_csv` in `config.json` to the small CSV.
3. Set `output_dir` to a temporary test folder.
4. Run the generator.
5. Open several generated images and compare them with the original autoscaled images.
6. Run the validation script.
7. After a successful test, delete the temporary output, point the configuration to the full CSV, and choose the final output folder.

9. Step 5 - Run the dataset generator

Option A: double-click

Double-click run_generator.bat.

Option B: terminal command

```
python generate_future_regime_dataset.py
```

Option C: virtual environment without activation

```
.venv\Scripts\python.exe generate_future_regime_dataset.py
```

During execution, the program will:

1. Load and clean the OHLC CSV.
2. Parse timestamps and sort the records chronologically.
3. Reject invalid OHLC rows and detect gaps in each 60-candle input-plus-target span.
4. Generate current and future autoscaled candlestick images.
5. Load pretrained ResNet-18 and the saved PCA and K-Means models.
6. Label each unique future image with a cluster ID and centroid-distance information.
7. Create chronological training, validation, and test partitions.
8. Hash all unique images to identify exact duplicates.
9. Write manifests, reports, summary files, and skipped-sample information.

10. Understanding the generated folder

```
supervised_future_regime_dataset/
├── images/
│   ├── sample_0_29.jpg
│   ├── sample_1_30.jpg
│   └── ...
├── manifests/
│   ├── all_samples.csv
│   ├── train_samples.csv
│   ├── validation_samples.csv
│   ├── test_samples.csv
│   ├── dataset_summary.json
│   └── duplicate_images.json
├── reports/
│   └── cluster_distribution.csv
└── skipped_samples.txt
```

10.1 Meaning of a manifest row

A representative row records:

- `sample_name`: identifier for the current input window.
- `input_start_index` and `input_stop_index`: t and $t+29$.
- `input_image`: image supplied to the future supervised classifier.
- `target_start_index` and `target_stop_index`: $t+30$ and $t+59$.
- `target_image`: future image used only to calculate the pseudo-label.
- `target_cluster_id`: supervised class label from the saved K-Means model.
- `target_regime_name`: provisional descriptive cluster name.
- `target_distance_to_centroid`: distance to the assigned cluster centroid.
- `target_second_nearest_distance` and `target_distance_margin`: measures useful for identifying ambiguous assignments.
- `target_distance_cluster_0` through `target_distance_cluster_3`: distances to every cluster.

10.2 Cluster mapping

Cluster	Provisional interpretation
0	High-volatility consolidation
1	Bullish trend and breakout
2	Volatile sideways and reversal
3	Bearish trend and breakdown

11. Step 6 - Validate the generated dataset

Option A: double-click

Double-click `validate_dataset.bat`.

Option B: terminal command

```
python validate_generated_dataset.py
```

The validator checks:

- all expected manifest and summary files exist;
- all `input_image` and `target_image` files referenced by the manifests exist;
- cluster counts are available for all partitions;
- the final training target ends before validation begins;
- the final validation target ends before testing begins.

A successful result ends with:

```
Leakage checks passed.
Dataset package appears valid.
```

12. Quality checks before supervised training

1. Open random generated images from early, middle, and late sections of the dataset.
2. Compare their size, colors, margins, candle width, wick width, background, and autoscaling with the original unsupervised images.
3. Review manifests/dataset_summary.json for sample counts, boundaries, skipped samples, and cluster distributions.
4. Review manifests/duplicate_images.json for exact duplicate groups.
5. Review reports/cluster_distribution.csv for class imbalance.
6. Inspect low-margin or high-distance labels before deciding whether to filter ambiguous samples.
7. Back up the completed dataset before beginning classifier development.

13. Optional filtering of uncertain pseudo-labels

The generator records centroid distances but does not automatically remove uncertain labels. This is deliberate because a defensible threshold should be chosen from the distribution of training distances rather than guessed.

- Small `target_distance_to_centroid` generally indicates a more typical target image.
- A small `target_distance_margin` means the closest and second-closest clusters are similar, so the label may be ambiguous.
- Distance values are not calibrated probabilities and should not be called confidence scores.
- Any filtering threshold should be documented and applied using only training-derived criteria.

14. Using the manifests for classifier training

The future supervised model should be trained using:

```
input_image -> target_cluster_id
```

Use `train_samples.csv` for model fitting, `validation_samples.csv` for hyperparameter selection and checkpointing, and `test_samples.csv` once for final out-of-sample evaluation. Do not use `target_image` as an input feature because it contains the future period being predicted.

15. Restarting after an interruption

The current package is designed for a clean full run and does not implement a resumable checkpoint system. If the process stops before completion, the safest approach is to delete the incomplete output folder and rerun. If `overwrite_output` is false and the output folder is non-empty, the generator will stop instead of overwriting it.

1. Check whether the output folder is incomplete.

2. Move it elsewhere if you want to inspect partial files.
3. Delete or rename the incomplete output folder.
4. Keep `overwrite_output=false` for safety.
5. Rerun the generator.

16. Troubleshooting

Problem	Action
OHLC CSV not found	Check the absolute <code>ohlc_csv</code> path and use doubled backslashes in <code>config.json</code> .
PCA or K-Means model not found	Confirm <code>pca_model</code> and <code>kmeans_model</code> point directly to the <code>.joblib</code> files.
Could not find a column	Print the CSV header and update the columns section of <code>config.json</code> .
Invalid <code>expected_frequency</code>	Use a pandas-style value such as 1h, 30min, or 4h.
Many timestamp-gap skips	The source may contain missing hours, duplicates, the wrong frequency, or incorrectly parsed timestamps.
Output directory is not empty	Choose a new output directory, remove the incomplete directory, or deliberately set <code>overwrite_output=true</code> .
CUDA out of memory	Reduce <code>batch_size</code> to 16, 8, or 4. Image generation itself runs on the CPU.
PyTorch uses CPU	This is acceptable. Predictions will be slower but valid.
ResNet weights fail to download	Connect to the internet once, then rerun. PyTorch normally caches the weights.
Images do not resemble the old autoscaled dataset	Stop the full run and adjust <code>chart_style</code> . Do not accept pseudo-labels from a visibly different chart style.
Validation split is empty	Reduce <code>extra_embargo_candles</code> , adjust split fractions, use a smaller stride, or supply more OHLC rows.
Duplicate report is large	Investigate repeated OHLC windows or chart-generation conditions before training.

17. Important methodological cautions

- The target classes are pseudo-labels generated by the earlier unsupervised model, not manually observed ground truth.
- The provisional regime names must remain distinct from the numeric cluster IDs in the data.
- Adjacent samples overlap by 29 candles when `stride=1`, so random train/test splitting would cause serious leakage.
- The saved PCA and K-Means models must not be refitted during supervised dataset generation.
- The chart-generation appearance should match the earlier autoscaled images as closely as possible.
- The future classifier predicts the visual regime of the next 30 candles, not a guaranteed return or trading outcome.

18. Full run checklist

- ☐ Package extracted successfully.
- ☐ Python dependencies installed.
- ☐ OHLC CSV path confirmed.
- ☐ PCA and K-Means files confirmed.
- ☐ Output folder selected.
- ☐ Column names checked.
- ☐ Small test run completed.
- ☐ Generated image style visually verified.
- ☐ Full generator completed.
- ☐ Validator passed.
- ☐ Cluster distributions reviewed.
- ☐ Dataset backed up.

19. Command summary

```
# Install
python -m pip install -r requirements.txt

# Generate
python generate_future_regime_dataset.py

# Validate
python validate_generated_dataset.py

# Using the package virtual environment without activation
.venv\Scripts\python.exe generate_future_regime_dataset.py
.venv\Scripts\python.exe validate_generated_dataset.py
```

20. Final expected outcome

After the generator and validator complete successfully, the package will have produced a leakage-controlled, chronologically divided supervised dataset. Each input chart represents the current 30 candles, while its `target_cluster_id` represents the unsupervised cluster assigned to the immediately following 30 candles. This dataset can then be used to train and evaluate a four-class future-regime image classifier.